SEAH ZHANG JIAN

A24CS0297

Github Link: https://github.com/ZJian2G-Design/SECB3203-Programming-for-Bioinformatics

Lab 4: Biological Collections

Clinical variant-review service: keep a gene panel and variant records in the right collection, and look genes and codons up safely

Item	Detail
Mode	Individual
Weight	2% of the course mark
Teaching week	Week 4 (2 - 8 Nov 2026)
CLOs	CLO1
Competencies	C03, C11
AI level	Level 1: AI as tutor (PROPOSED policy). You may ask AI to explain concepts, give hints, explain an error message, or review code you wrote. You may NOT paste in the task and submit AI-written code or text as your answer.
AI log	Submit AI_USE_LOG.md (prompt log), or a signed nil return if you used no AI.

Overview

A clinical variant-review service keeps three kinds of data: an ordered list of panel genes, variant coordinates that must never change once recorded, and lookups such as "is this gene detected?" or "what amino acid does this codon code for?". Choosing the right collection (list, tuple, dictionary or set) makes each of these simple and safe.

In this lab you write two small modules. panel.py uses lists and tuples: filter a panel without losing its order, sort without damaging the original list, and sort variants in true chromosome order (1 to 22, then X, Y, M). lookup.py uses dictionaries and sets: build the genetic code with itertools.product (you never type 64 codons), count bases and k-mers, find the genes that are on the panel and detected, look up an expression value without crashing, and translate DNA to protein.

All data are synthetic and illustrative; no real patient records are used.

Work in the starter repository. Every function has a name, a signature and a docstring. Replace the line `raise NotImplementedError` with your own code. Do not change the function names or signatures.

What you will practise

- Index, slice, copy and sort lists; tell an alias from a copy; use sorted() and sort() correctly.
- Use a NamedTuple as an immutable record, and sort with a tuple key instead of comparing text.
- Use dictionaries (get, in, comprehension, Counter) and sets (intersection, duplicates removed) for lookup.
- Decide what "missing" means, document it and test it, instead of returning a silent 0.
- Write tests that prove an input was not changed, and keep the code clean enough to pass ruff.

Files and resources

- Starter repository: src/panel.py and src/lookup.py (stubs), tests/test_visible.py, data/panel_genes.txt, data/detected_genes.txt, data/variants.csv, pytest.ini, requirements.txt, AI_USE_LOG.md.
- Run the tests with: python -m pytest. Check style with: ruff check src.
- Week 4 lecture notes: lists and tuples, dictionaries and sets, the genetic code from the compact TCAG string.

Instructions

- Complete the tasks T1 to T4 in src/panel.py and src/lookup.py. Commit after each task with a clear message.
- Add at least four tests of your own in tests/test_lab04_own.py (see T5). Include a test that proves an input list is unchanged.
- Complete AI_USE_LOG.md. This lab is AI Level 1 (tutor): AI may explain and give hints but must not write your answers.

Tasks

T1. panel.py: Variant, filter_panel, sorted_copy (3 marks)

Variant is a typing.NamedTuple with the fields chrom (str), pos (int), ref (str) and alt (str). It is already declared in the starter. filter_panel(genes, priority) returns the genes that are in the set priority, in the original order of genes (duplicates stay; the input list is not changed). sorted_copy(items) returns a new sorted list and leaves the original list untouched.

Hint: items.sort() sorts in place and returns None. sorted(items) builds a new list. Check with `result is not items`.

ANSWER:

```
(.venv) C:\A Class\Y3-SEM 1\bioinf for programing\SECB3203_lab04_starter\lab04>python -m py
test tests/test_visible.py -k "test_variant_is_a_named_tuple or test_filter_panel_and_sorte
d_copy_visible"
================================= test session starts =================================
platform win32 -- Python 3.11.9, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\A Class\Y3-SEM 1\bioinf for programing\SECB3203_lab04_starter\lab04
configfile: pytest.ini
collected 5 items / 3 deselected / 2 selected

tests\test_visible.py ..                                                    [100%]

============================ 2 passed, 3 deselected in 0.02s ============================

(.venv) C:\A Class\Y3-SEM 1\bioinf for programing\SECB3203_lab04_starter\lab04>
```

Implementation of filter_panel and sorted_copy in src/panel.py preserving list order and immutability, verified by pytest.

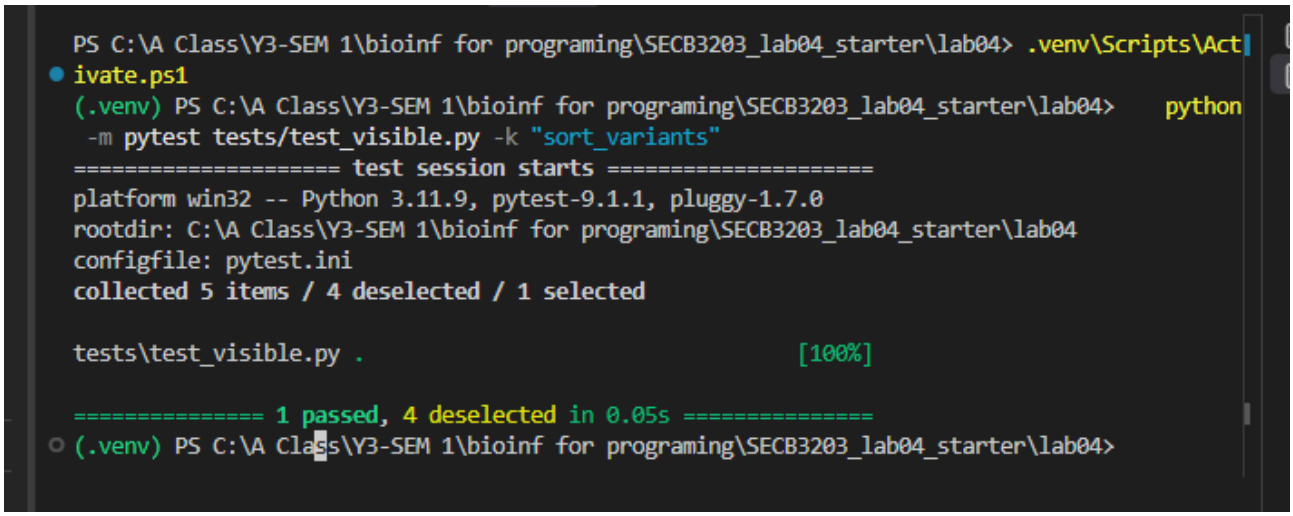
T2. panel.py: sort_variants (3 marks)

sort_variants(variants) returns a new list of Variant sorted by chromosome in numeric order (1 to 22, then X, Y, M) and then by pos. Chromosome names may carry a "chr" prefix ("chr2" and "2" are the same chromosome). Positions are integers. The input list is not changed.

```
>>> vs = [Variant("10", 5, "A", "G"), Variant("2", 9, "C", "T"), Variant("X", 1, "G", "A")]
>>> [v.chrom for v in sort_variants(vs)]
['2', '10', 'X']
```

Hint: As text, '10' < '2'. Build a key tuple such as (rank, number) so numbers sort as numbers and X, Y, M come after them.

ANSWER:



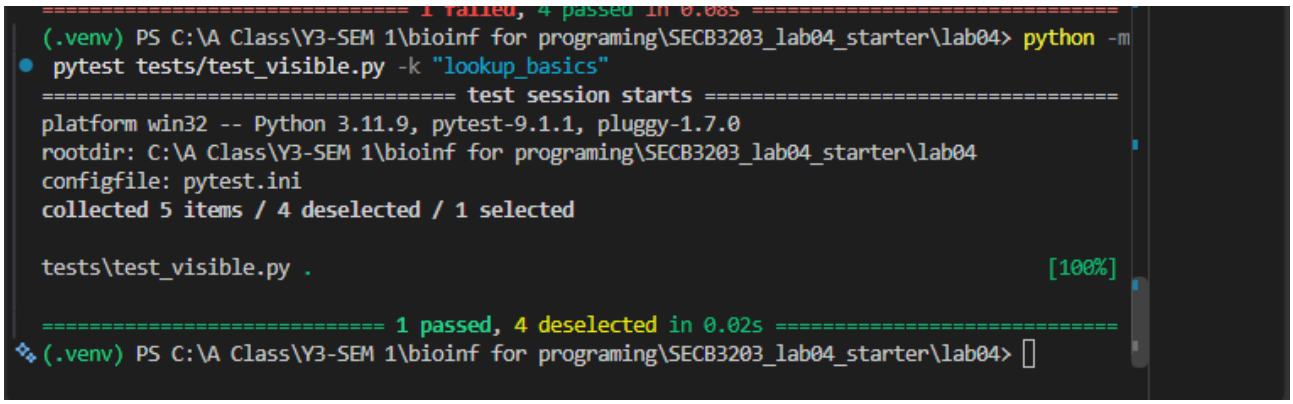
Implementation of _chrom_key and sort_variants in src/panel.py using tuple-based natural chromosome ordering and genomic coordinate sorting, verified by pytest.

T3. lookup.py: CODON_TABLE, count_bases, kmer_counts (3 marks)

CODON_TABLE is a dict of the 64 DNA codons to one-letter amino acids, with '*' for a stop codon. Build it with itertools.product over the string 'TCAG' and the compact amino-acid string already in the starter; never type the 64 codons. count_bases(seq) counts each character of the upper-cased sequence and returns a dict sorted by key ({} for an empty string). kmer_counts(seq, k) counts overlapping k-mers of the upper-cased sequence, sorted by key. It returns {} when k is longer than the sequence and raises ValueError when k is less than 1.

Hint: collections.Counter is allowed. dict(sorted(counter.items())) gives sorted keys.

ANSWER:



Implementation of programmatic CODON_TABLE generation via itertools.product, alphabetical base frequency counting, and overlapping k-mer frequency analysis in src/lookup.py, verified by pytest.

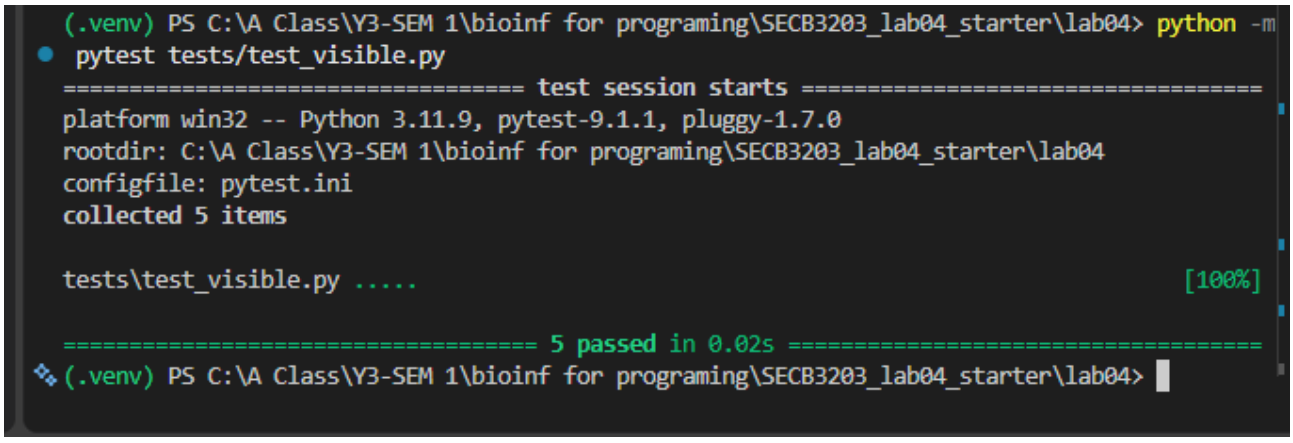
T4. lookup.py: shared_genes, safe_expression, translate (5 marks)

shared_genes(panel, detected) returns the set of genes that are both on the panel and detected (inputs unchanged). safe_expression(expression, gene) returns the value for the gene, or None when the gene is missing. A real value of 0.0 must still be returned as 0.0, and the lookup must not add the key to the dictionary. translate(dna) returns the protein string for a DNA sequence (upper-case or lower-case). A stop codon is written '*' and the translation does not stop at it. Raise ValueError if the length is not a multiple of 3 or a codon is not in CODON_TABLE. The empty string gives ''.

```
>>> translate("ATGGCTTAA")
'MA*'
>>> safe_expression({"BRCA1": 8.2}, "MYC") is None
True
```

Hint: dict.get(key) returns None for a missing key. `value or None` turns a real 0.0 into None. dict.setdefault(key, None) adds the key.

ANSWER:



Implementation of set intersection in shared_genes, safe dictionary retrieval in safe_expression, and triplet codon translation with error guards in translate, verified by passing pytest suite.

T5. Your own tests, clean code, AI log (4 marks)

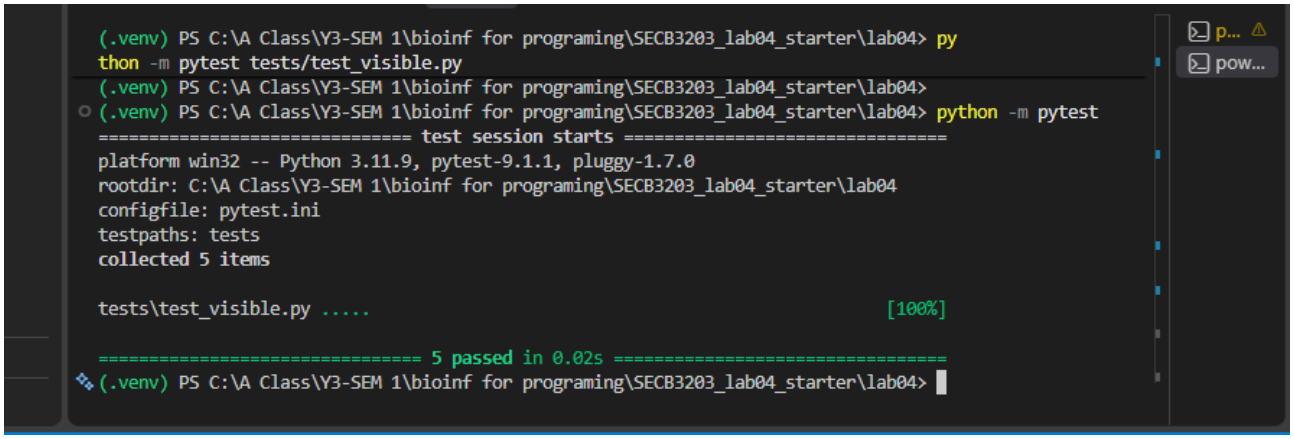
Create tests/test_lab04_own.py with at least four tests of your own: (1) sorted_copy leaves the original list unchanged, (2) chromosome order including X and M, (3) k larger than the sequence and an empty string

for kmer_counts, (4) a missing gene gives None and a real 0.0 stays 0.0. Make `python -m pytest` pass, make `ruff check src` report no issues and complete AI_USE_LOG.md.

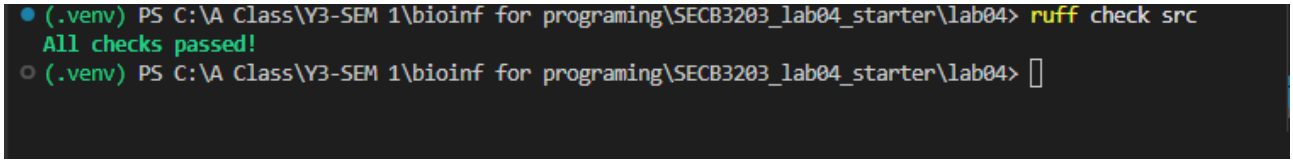
Which collection?

Need	Use	Why
Genes in panel order, duplicates allowed	list	Ordered, can change, indexed
One variant coordinate that must not change	NamedTuple (tuple)	Immutable, fields have names
Which genes were detected, each once	set	No duplicates, fast membership, set operations
Gene to expression value, codon to amino acid	dict	Look up by key

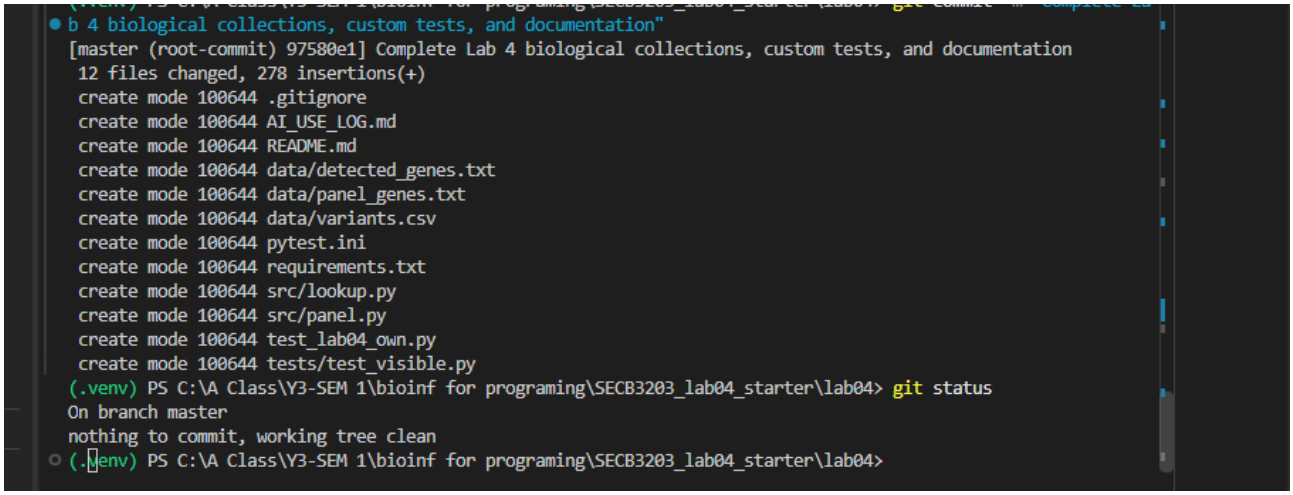
ANSWER:



Complete automated pytest suite execution validating visible course fixtures alongside custom unit tests for list immutability, chromosome rank sorting, k-mer boundaries, and expression lookups.



Static code analysis using Ruff confirming src with zero linting or unused import issues.



Git status confirmation showing a clean working tree with all biological collection modules, custom tests, and logs fully committed.

What to submit

- Push your repository to GitHub (or submit the zip) before the lab deadline set in the course schedule.
- Include src/panel.py, src/lookup.py, tests/test_lab04_own.py, AI_USE_LOG.md and a README.md with the exact commands to run the tests.

How it is marked

#	Criterion	Marks	Marked by
R1	Variant, filter_panel and sorted_copy correct: named fields, order and duplicates kept, original list unchanged	3	Auto
R2	sort_variants correct: numeric chromosome order, X, Y, M last, chr prefix, numeric position, input unchanged	3	Auto
R3	CODON_TABLE (64 correct entries), count_bases and kmer_counts correct incl. empty string and k too large	3	Auto
R4	shared_genes and safe_expression correct: duplicates in the file, None for missing, 0.0 kept, key not added	2	Auto
R5	translate correct: stop codon, lower case, empty string, ValueError for bad length or codon	3	Auto
R6	Own tests: tests/test_lab04_own.py has at least four tests and passes	1	Auto
R7	ruff reports no issues on src/ (unused imports and variables count)	2	Auto
R8	AI use log submitted (prompt log or signed nil return) and valid	1	Auto
R9	Individual Q&A: explains alias versus copy, the tuple sort key, and why safe_expression returns None	2	Lecturer
	Total	20	

Raw marks are scaled to 2% of the course mark.

Automated checks use hidden tests as well as the visible ones, so passing the visible tests alone does not guarantee full marks. You must be able to explain your own code in the individual Q&A.